МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине

«Проектирование вычислительных систем»

Вариант: 5

Работу выполнили:
Студенты группы P3411
Болорболд Аригуун
Кравцов Кирилл Денисович

Преподаватель:
Пинкевич Василий Юрьевич

г. Санкт-Петербург
2025

Содержание

1. Текст задания 3
2. Выполнение 5
   2.1. Распиновка 5
   2.2. Блок-схема: 6
   2.3. Описание работы алгоритма 6
   2.4. Код драйвера 7
3. Вывод 13

1. Текст задания

Разработать и реализовать два варианта драйверов UART для стенда SDK-1.1M: с использованием и без использования прерываний. Драйверы, использующие прерывания, должны обеспечивать работу в «неблокирующем» режиме (возврат из функции происходит сразу же, без ожидания окончания приема/отправки), а также буферизацию данных для исключения случайной потери данных. В драйвере, не использующем прерывания, функция приема данных также должна быть «неблокирующей», то есть она не должна зависать до приема данных (которые могут никогда не поступить). При использовании режима «без прерываний» прерывания от соответствующего блока UART должны быть запрещены.

Написать с использованием разработанных драйверов программу, которая выполняет определенную вариантом задачу. Для всех вариантов должно быть реализовано два режима работы программы: с использованием и без использования прерываний. Каждый принимаемый стендом символ должен отсылаться обратно, чтобы он был выведен в консоли (так называемое «эхо»). Каждое новое сообщение от стенда должно выводиться с новой строки. Если вариант предусматривает работу с командами, то на каждую команду должен выводиться ответ, определенный в задании или «ОК», если ответ не требуется. Если введена команда, которая не поддерживается, должно быть выведено сообщение об этом.

Доработать программу кодового замка. Теперь ввод кода должен происходить не с помощью кнопки стенда, а по UART. После ввода единственно верной последовательности из восьми латинских букв без учёта регистра и цифр должен загореться зелёный светодиод, обозначающий «открытие» замка. Светодиод горит некоторое время, потом гаснет, и система вновь переходит в «режим ввода». Каждый неправильно введённый последовательности должен сопровождаться миганием красного светодиода и сбросом в «начало», каждый правильный – миганием жёлтого. После трёх неправильных вводов начинает мигать красный светодиод, и через некоторое время система вновь возвращается в «режим ввода». Если код не введен до конца за некоторое ограниченное время, происходит сброс в «начало».

Должно быть предусмотрено изменение отпирающей последовательности, что производится следующей последовательностью действий:

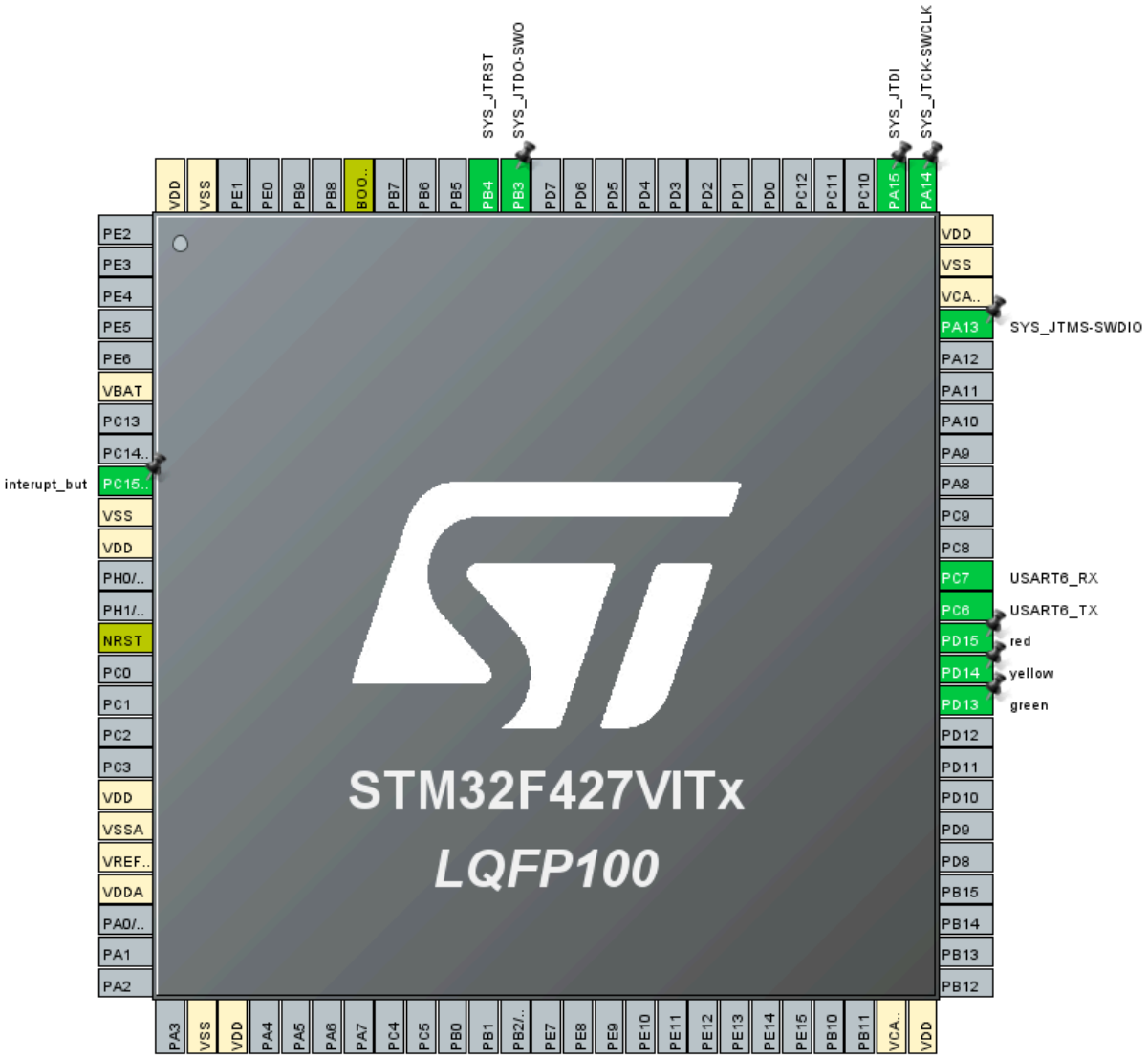
- ввод символа «+»;

- ввод новой последовательности, который завершается либо по нажатию enter, либо по достижении восьми значений;
- стенд отправляет сообщение произвольного содержания, спрашивая, сделать ли последовательность активной, и запрашивает подтверждение, которое должно быть сделано вводом символа y;
- после ввода y введённая последовательность устанавливается как активная.

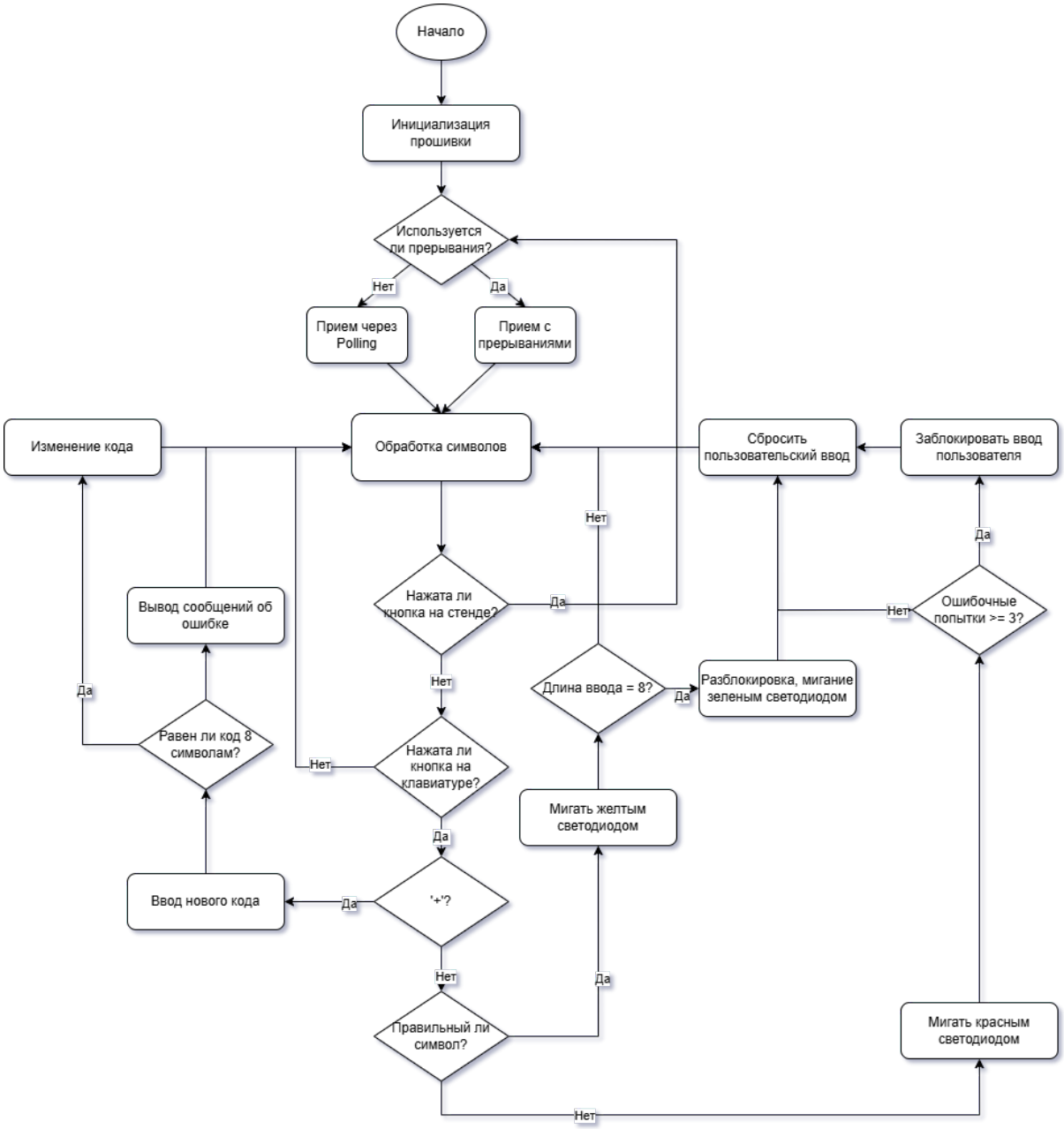
Включение/отключение прерываний должно осуществляться нажатием кнопки на стенде и сопровождаться отправкой в последовательный порт сообщения произвольного содержания, сообщающего, какой режим включен (с прерываниями или без прерываний). Скорость обмена данными по UART – 9600 бит/с.

2. Выполнение

2.1. Распиновка



2.2. Блок-схема:



2.3. Описание работы алгоритма

Надо в терминале выбрать правильный порт и установить соответствующую скорость передачи данных. После этого появится сообщение на экране, после чего можно будет ввести код через клавиатуру устройства (не стенда). Если встретится хоть один символ, то пользовательский ввод сбрасывается и на стенде мигает светодиод красным цветом. Если пользователь ошибается трижды подряд, то происходит «блокировка» стенда и пользователю запрещается ввод в течении 5 секунд. Если один символ правильный, то — желтым, а если все символы правильные, то происходит «разблокировка» и подсвечивает зеленым цветом, после чего все (ввод, попытки до блокировки)

возвращается в исходное состояние. При ввода символа плюса «+» пользователю предлагается ввести новый код. Если пользователь ввел код длиной менее 8 символов, то ввод сбрасывается. Если равно, то пользователю предлагается подтвердить введенный код. Символы нового кода заменяется звездочками «*». Также при нажатии на белую кнопку стенда программа переходит на режим прерывании.

2.4. Код драйвера

```
#include "main.h"
#include "usart.h"
#include "gpio.h"

#include <string.h>
#include <ctype.h>

/* USER CODE BEGIN PD */
#define CODE_LENGTH 8
#define MAX_ATTEMPTS 3
#define TIMEOUT_MS 10000
#define GREEN_LED_TIME 3000
#define RED_BLINK_TIME 5000
char current_code[CODE_LENGTH + 1] = "12345678";
char input_buffer[CODE_LENGTH + 1] = {0};
char new_code_buffer[CODE_LENGTH + 1] = {0};
uint8_t input_pos = 0;
uint8_t failed_attempts = 0;
uint32_t last_input_time = 0;
uint8_t setting_new_code = 0;
uint8_t new_code_pos = 0;
uint8_t use_interrupts = 0;
uint8_t rx_byte = 0;


volatile uint8_t char_received = 0;
volatile char received_char = 0;

void SystemClock_Config(void);
void turn_on_green_led(void);
void turn_on_yellow_led(void);
void turn_on_red_led(void);
void turn_off_green_led(void);
void turn_off_yellow_led(void);
void turn_off_red_led(void);
void turn_off_all_leds(void);
void blink_led(GPIO_TypeDef* port, uint16_t pin, uint16_t
```

```
duration);
void send_string(const char* str);
void reset_input(void);
void check_timeout(void);
void process_char(char c);
void lock_system(void);
void unlock_system(void);
static void wait_ms(uint32_t ms);

static void wait_ms(uint32_t ms) {
    uint32_t start = HAL_GetTick();
    while ((HAL_GetTick() - start) < ms) {
        __NOP();
    }
}

void turn_on_green_led(void) {
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_SET);
}

void turn_on_yellow_led(void) {
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_SET);
}

void turn_on_red_led(void) {
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_SET);
}

void turn_off_green_led(void) {
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_RESET);
}

void turn_off_yellow_led(void) {
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_RESET);
}

void turn_off_red_led(void) {
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_RESET);
}

void turn_off_all_leds(void) {
    turn_off_green_led();
    turn_off_yellow_led();
    turn_off_red_led();
}
```

```c
void blink_led(GPIO_TypeDef* port, uint16_t pin, uint16_t duration)
{
    HAL_GPIO_WritePin(port, pin, GPIO_PIN_SET);
    wait_ms(duration);
    HAL_GPIO_WritePin(port, pin, GPIO_PIN_RESET);
}

void send_string(const char* str) {
    HAL_UART_Transmit(&huart6, (uint8_t*)str, strlen(str), 1000);
}

void reset_input(void) {
    input_pos = 0;
    memset(input_buffer, 0, sizeof(input_buffer));
    last_input_time = HAL_GetTick();
}

void check_timeout(void) {
    if (input_pos > 0 && (HAL_GetTick() - last_input_time >
TIMEOUT_MS)) {
        send_string("Timeout! Code reset.\r\n");
        blink_led(GPIOD, GPIO_PIN_14, 1000);
        reset_input();
    }
}

void lock_system(void) {
    send_string("!!! LOCKED !!! Too many attempts.\r\n");
    uint32_t start = HAL_GetTick();
    while ((HAL_GetTick() - start) < RED_BLINK_TIME) {
        turn_on_red_led();
        wait_ms(250);
        turn_off_red_led();
        wait_ms(250);
    }
    failed_attempts = 0;
    reset_input();
    send_string("System ready. Enter code:\r\n");
}

void unlock_system(void) {
    send_string("\r\n*** UNLOCKED! ***\r\n");
    turn_on_green_led();
    wait_ms(GREEN_LED_TIME);
    turn_off_green_led();
    failed_attempts = 0;
```

```
    reset_input();
    send_string("System ready. Enter code:\r\n");
}

void process_char(char c) {
    if (setting_new_code == 1) {
        if (c == '\r' || c == '\n') {
            if (new_code_pos >= CODE_LENGTH) {
                send_string("\r\nNew code entered. Make it active?
(y/n): ");
                setting_new_code = 2;
            } else {
                send_string("\r\nCode too short! Need 8 characters.
\r\n");
                setting_new_code = 0;
                new_code_pos = 0;
                memset(new_code_buffer, 0,
sizeof(new_code_buffer));
            }
            return;
        }

        if (isalnum((unsigned char)c) && new_code_pos <
CODE_LENGTH) {
            new_code_buffer[new_code_pos++] = tolower((unsigned
char)c);
            HAL_UART_Transmit(&huart6, (uint8_t*)"*", 1, 100);

            if (new_code_pos >= CODE_LENGTH) {
                send_string("\r\nNew code entered. Make it active?
(y/n): ");
                setting_new_code = 2;
            }
        }
        return;
    }

    if (setting_new_code == 2) {
        if (c == 'y' || c == 'Y') {
            strcpy(current_code, new_code_buffer);
            send_string("\r\nNew code activated!\r\n");
            blink_led(GPIOD, GPIO_PIN_13, 500);
        } else {
            send_string("\r\nNew code cancelled.\r\n");
        }
        setting_new_code = 0;
```

```
        new_code_pos = 0;
        memset(new_code_buffer, 0, sizeof(new_code_buffer));
        reset_input();
        send_string("System ready. Enter code:\r\n");
        return;
    }


    if (c == '+') {
        send_string("\r\nEnter new code (8 chars): ");
        setting_new_code = 1;
        new_code_pos = 0;
        memset(new_code_buffer, 0, sizeof(new_code_buffer));
        return;
    }

    if (isalnum((unsigned char)c)) {
        c = tolower((unsigned char)c);
        input_buffer[input_pos] = c;
        last_input_time = HAL_GetTick();


        if (input_buffer[input_pos] == current_code[input_pos]) {
            blink_led(GPIOD, GPIO_PIN_14, 100);
            HAL_UART_Transmit(&huart6, (uint8_t*)&c, 1, 100);
            input_pos++;
            if (input_pos >= CODE_LENGTH) {
                unlock_system();
            }
        } else {
            send_string("\r\nWrong character!\r\n");
            blink_led(GPIOD, GPIO_PIN_15, 200);
            failed_attempts++;
            reset_input();

            if (failed_attempts >= MAX_ATTEMPTS) {
                lock_system();
            } else {
                send_string("Try again. Enter code:\r\n");
            }
        }
    }
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART6) {
```

```
        received_char = (char)rx_byte;
        char_received = 1;

        HAL_UART_Receive_IT(&huart6, &rx_byte, 1);
    }
}

int main(void)
{
  uint8_t button_pressed = 0;
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_USART6_UART_Init();

  turn_off_all_leds();

  HAL_NVIC_SetPriority(USART6_IRQn, 0, 0);
  HAL_NVIC_EnableIRQ(USART6_IRQn);

  send_string("\r\n");
  send_string("==========================================\r\n");
  send_string("    CODE LOCK SYSTEM v2.0 (UART)\r\n");
  send_string("==========================================\r\n");
  send_string("Mode: POLLING (press button to toggle)\r\n");
  send_string("System ready. Enter code:\r\n");

  reset_input();
  while (1)
  {
      if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_15) == GPIO_PIN_RESET
&& !button_pressed) {
          wait_ms(50);
          if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_15) ==
GPIO_PIN_RESET) {
              button_pressed = 1;
              use_interrupts = !use_interrupts;

              if (use_interrupts) {
                  send_string("\r\n>>> Mode: INTERRUPTS <<<\r\n");
                  HAL_UART_Receive_IT(&huart6, &rx_byte, 1);
              } else {
                  send_string("\r\n>>> Mode: POLLING <<<\r\n");
                  HAL_UART_AbortReceive_IT(&huart6);
              }
```

```
                blink_led(GPIOD, GPIO_PIN_13, 200);
            }
        }

        if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_15) == GPIO_PIN_SET) {
            button_pressed = 0;
        }


        if (!use_interrupts) {
            char c;
            if (HAL_OK == HAL_UART_Receive(&huart6, (uint8_t*)&c, 1,
10)) {
                process_char(c);
            }
        } else {
            if (char_received) {
                char_received = 0;
                process_char(received_char);
            }
        }
        check_timeout();
        wait_ms(10);
    }
}
```

3. Вывод

В ходе выполнения работы мы доработали кодовый замок на основе неблокирующего обмена через интерфейс UART в режиме прерываний и без, а также с использованием кольцевого буфера. Сигнализировали о неверном значений символа с помощью красного светодиода, а при верном — желтого, и при правильном последовательности — зеленым.